Natural Language Processing - Lecture 02

Lecture overview

Lecture 02 establishes tokenization as the foundational process of breaking text into smaller units, necessitating a preceding normalization step for unclean data such as tweets containing emojis or slang. The lecture details Regular Expressions (Regex) as a formal language for defining these patterns, covering core syntax including character classes, quantifiers, anchors, and advanced features like group capturing, backreferences, and lookahead assertions. A critical exam concept highlighted is the iterative nature of pattern design, which involves managing the trade-off between recall (minimizing false negatives) and precision (minimizing false positives), illustrated by the progression from simple word matching to isolating full words using non-word character boundaries. While Regex serves as a vital pre-processing tool for tasks like sentence segmentation-where ambiguity in periods due to abbreviations must be resolved-it has inherent limitations in handling complex structures like URLs or emails, exemplified historically by the ELIZA chatbot which relied on substitution patterns without true understanding.

The lecture further transitions to word normalization and subword tokenization, distinguishing lemmatization (reducing inflected words to a root lemma) from the data-driven approach of Byte Pair Encoding (BPE). BPE is described as an iterative algorithm that starts with individual characters and repeatedly merges the most frequent adjacent symbol pairs until a target vocabulary size is reached, creating tokens based on frequency rather than linguistic meaning. This method balances vocabulary size by avoiding out-of-vocabulary issues while representing frequent sequences efficiently, though it faces challenges regarding morphological complexity and multilingual bias, where non-English languages often result in longer token sequences. The session concludes by positioning Regex as essential for pre-processing before subword tokenization and noting that tokenization remains an active research field with ongoing developments in cross-lingual fairness and alternative methods like pixel-based models.

Slide 1: Natural Language Processing:

This lecture introduces the topic of Natural Language Processing, specifically focusing on tokenization. The course is identified as H02B1A at the Department of Computer Science, KU Leuven, and is presented by Miryam de Lhoneux. The presentation materials are adapted from Jurafsky and Martin (2024).

Slide 2: Introduction

This slide introduces the lecture outline on Natural Language Processing (NLP) tokenization. The session covers regular expressions, basics, and applications before detailing word tokenization, normalization, and sentence segmentation. It further explores subword tokenization methods like Byte Pair Encoding (BPE), discusses their limitations, and concludes with a summary.

Slide 3: Introduction

Slide 3 introduces a toy corpus for Natural Language Processing (NLP) tokenization, defining a specific vocabulary including sky, wings, engine, helicopter, drone, rocket, goose, eagle, and bee. The slide presents various sentences describing these entities' relationships and attributes to illustrate how vocabulary is determined within this context. This example serves as the foundational dataset for exploring tokenization concepts in the lecture.

Slide 4: Introduction

Real-world text data, such as tweets containing emojis, hashtags, slang like "wanna," and abbreviations like "MT" or "HMU," is often unclean. Consequently, this raw input typically requires a normalization step before processing. This slide introduces tokenization as the third key concept in Natural Language Processing (NLP) for handling such data.

Slide 5: Introduction

Tokenization converts a text string into a sequence of units for later processing. What counts as a token depends on the task and language: spaces do not solve punctuation, contractions, compounds, or languages without whitespace word boundaries. Subword tokenization provides another option when word forms are rare or unseen.

Slide 6: Regular expressions

Slide 6 presents the outline for the lecture on regular expressions within the context of NLP tokenization. The agenda covers an introduction, basics, and applications of regular expressions, followed by detailed sections on word tokenization, normalization, sentence segmentation, and subword tokenization including BPE and its limitations. The slide concludes with a summary of these topics under the broader theme of NLP tokenization.

Slide 7: Regular expressions

Slide 7 presents the outline for the lecture on regular expressions within the context of NLP tokenization. The agenda covers an introduction, basics, and applications of regular expressions, followed by detailed sections on word tokenization, normalization, sentence segmentation, and subword tokenization including BPE and its limitations. The slide concludes with a summary of these topics under the broader theme of NLP tokenization.

Slide 8: Regular expressions

Regular expressions are defined as a formal language used for specifying text strings. In the context of Natural Language Processing (NLP), they are specifically applied to tokenization, which involves breaking text into smaller units. This slide establishes regular expressions as a foundational tool for defining patterns within textual data.

Slide 9: Regular expressions

Regular expressions provide a method to search for specific text patterns, such as mentions of cute animals like "woodchuck" and "groundhog." This approach is essential for handling variations in spelling and capitalization, including singular and plural forms (e.g., "Woodchucks" or "groundhogs"). The slide introduces this concept within the broader context of NLP tokenization.

Slide 10: Regular expressions

Regular expressions efficiently capture text variations using disjunctions within brackets, such as `[wW]` to match either 'w' or 'W'. Ranges defined by a dash, like `[0-9]`, provide a concise way to match any single digit, while `[A-Z]` and `[a-z]` target uppercase and lowercase letters respectively. Additionally, placing a caret inside brackets negates the pattern, matching the first character that is not included in the specified set.

Slide 11: Regular expressions

Inside square brackets, special characters like `.`, `*`, `+`, and `?` lose their special meaning and match literally. The caret (`^`) acts as a negation operator only when placed as the first character within the brackets, such as in `[^A-Z]` to exclude uppercase letters. If the caret appears elsewhere in the list, like `[e^]`, it is treated simply as a literal character to match.

Slide 12: Regular expressions

Slide 12 defines regular expression patterns as convenient aliases for common character classes used in NLP tokenization. The slide lists specific pairs where `\d` matches any digit, `\w` matches alphanumeric characters or underscores, and `\s` matches whitespace, alongside their negated counterparts `\D`, `\W`, and `\S`. Examples provided include "Latitude 59" for digits, "Blue moon" for non-digits, "Daiyu" for alphanumerics, and "Look!" for non-alphanumeric characters.

Slide 13: Regular expressions

Regular expressions utilize the pipe symbol (|) for disjunction to match alternative patterns, such as "groundhog" or "woodchuck," at either the word or character level. The syntax supports case-insensitive matching using brackets like `[gG]` and includes special characters for digits (`\d`), alphanumeric text (`\w`), and whitespace. These tools collectively capture variations in tokenization, allowing a single expression to handle multiple equivalent terms or formatting differences.

Slide 14: Regular expressions

Regular expressions utilize wildcards, optionality, and repetition to define matching patterns. The dot (.) matches any single character, while the question mark (?) makes the preceding character optional. Additionally, the asterisk (*) denotes zero or more occurrences of the previous character, and the plus sign (+) requires one or more occurrences.

Slide 15: Regular expressions

Regular expressions utilize specific patterns to match text, such as `^[A-Z]` for strings starting with an uppercase letter like "Leuven" or `^[^A-Za-z]` for non-alphabetic characters like "1". The symbols `.` and `$` represent any character and the end of a string respectively, while `^` and `$` function as anchors to define the beginning and end of a match. These patterns are fundamental tools in Natural Language Processing tasks, specifically for tokenization.

Slide 16: Regular expressions

Slide 16 presents an outline for the topic of Regular Expressions within the context of NLP tokenization. The agenda covers an introduction, basics, and applications, specifically detailing word tokenization, normalization, sentence segmentation, and subword tokenization including BPE and its limitations. This slide serves as a structural overview leading to a summary of these concepts.

Slide 17: Regular expressions

Defining regular expressions is an iterative process illustrated by finding all instances of "the." Starting with `the` misses capitalized forms, prompting a change to `[tT]he`, which incorrectly matches substrings like "other" or "Theology." The final solution, `\W[tT]he\W`, ensures the pattern matches only the full word by requiring non-word characters on both sides.

Slide 18: Regular expressions

Defining regular expressions involves an iterative process focused on correcting two specific errors: false negatives and false positives. False negatives occur when the expression fails to match intended strings, while false positives happen when it incorrectly matches unintended strings like "there" or "then." The professor notes that minimizing one error type often antagonistically increases the other, creating a trade-off between coverage and precision.

Slide 19: Regular expressions

Regular expressions involve balancing two antagonistic efforts to reduce error rates: increasing coverage (recall) to minimize false negatives and increasing accuracy (precision) to minimize false positives. This trade-off between recall and precision is characteristic of much work in Natural Language Processing. Students unfamiliar with these metrics, including F1, are directed to supplementary materials for further explanation.

Slide 20: Regular expressions

Regular expressions are used by various LLMs for NLP tokenization but face challenges with specific patterns like URLs, email addresses, prices, and abbreviations such as "PHD." Additional difficulties include handling clitics (e.g., the "'re" in "we're") and deciding whether multi-word expressions like "New York" should be treated as single tokens. Because regex alone is often insufficient and language-specific, it typically requires supplementing with explicit abbreviation lists to effectively capture these patterns.

Slide 21: Regular expressions

Regular expressions facilitate text substitution, such as converting American English to British English, using tools like Python or UNIX commands. The standard syntax follows the pattern `s/regexp1/pattern/`, exemplified by replacing "color" with "colour". This slide identifies regular expressions as a key component in Natural Language Processing tasks, specifically for tokenization.

Slide 22: Regular expressions

Group capturing uses parentheses to store matched patterns into numbered registers, allowing references like `\1` in replacements; for example, `s/([0-9]+)/<\1>/` wraps numbers in angle brackets. To avoid register clutter when grouping is unnecessary, adding a question mark after parentheses creates a non-capturing group that excludes the content from the register sequence.

Slide 23: Regular expressions

Regular expressions utilize group capturing with multiple registers to store matched substrings for later reference within the same pattern. In the example `/the (.*)er they (.*), the \1er we \2/`, the first register captures "fast" and the second captures "ran," allowing the expression to match "the faster they ran, the faster we ran." The pattern fails to match sentences like "the faster they ran, the faster we ate" because the backreference `\2` requires the final word to be identical to the captured "ran."

Slide 24: Regular expressions

Non-capturing groups are created by adding `?:` after an opening parenthesis, preventing that group from being assigned a register number for backreferences. In the example `/ (?:some|a few) (people|cats) like some \1/`, the first group is ignored, so `\1` refers only to the second group's match (`people` or `cats`). Consequently, "some cats like some cats" matches because the backreference aligns with "cats," whereas "some cats like some some" fails because "some" does not match the captured group.

Slide 25: Regular expressions

Regular expressions utilize lookahead assertions to match patterns based on subsequent characters without advancing the character pointer, known as zero-width matching. The negative lookahead `(?!pattern)` matches only if the pattern does not follow, such as `/q(?!u)/` finding 'q' in "faqir" but not "question". Conversely, the positive lookahead `(?=pattern)` matches when the pattern is present, allowing 'q' to be matched in "question" while excluding it from "faqir".

Slide 26: Regular expressions

Regular expressions serve as a foundational application in natural language processing, exemplified by ELIZA, an early chatbot that functioned as a therapist. ELIZA operated as a "giant regular expression" by matching user input against specific patterns to generate responses; for instance, it would replace statements like "I'm depressed" with empathetic replies such as "I'm sorry that you are depressed." This demonstrates how pattern-based replacement rules can simulate conversation without complex understanding.

Slide 27: Regular expressions

Slide 27 illustrates regular expression applications within the ELIZA program using substitution patterns. The examples demonstrate matching user inputs containing keywords like "depressed," "sad," "all," or "always" and replacing them with specific responses, utilizing backreferences (e.g., \1) to echo captured emotional states. These patterns serve as a practical demonstration of text processing techniques relevant to Natural Language Processing tokenization.

Slide 28: Word tokenzation and normalization and sentence segmentation

Slide 28 outlines the core components of text preprocessing, specifically word tokenization, word normalization, and sentence segmentation. The section further introduces subword tokenization techniques, including Byte Pair Encoding (BPE), while noting their associated limitations. This content serves as a structural overview within the broader NLP tokenization module.

Slide 29: Word tokenzation and normalization and sentence segmentation

This slide outlines the key components of text preprocessing in NLP, specifically word tokenization, word normalization, and sentence segmentation. It further introduces subword tokenization techniques, including Byte Pair Encoding (BPE), while noting their associated limitations. These concepts form the structural foundation for the subsequent detailed discussion on tokenization methods within the lecture.

Slide 30: Word tokenzation and normalization and sentence segmentation

Word tokenization involves preparing a toy corpus by adding spaces around punctuation marks, such as periods and commas, using the substitution pattern `s/([\.,])/ \1 /g`, or alternatively removing them entirely. Once spacing is adjusted, the text is split on white spaces to generate individual tokens. This process serves as a foundational step in natural language processing for segmenting raw text into manageable units.

Slide 31: Word tokenzation and normalization and sentence segmentation

Word tokenization involves splitting text into units, but blindly removing punctuation creates significant problems for specific formats. Critical examples where standard removal fails include hashtags like #nlproc, dates such as 8/09, URLs like kuleuven.be, abbreviations including Ph.D., prices like 2189.54€, and email addresses such as someone@kuleuven.be. These cases demonstrate that punctuation is often integral to the meaning or structure of the token rather than a separator.

Slide 32: Word tokenzation and normalization and sentence segmentation

Word tokenization faces challenges with clitics, which are words that cannot stand alone, such as "are" in "we're," French "je" in "j'ai," or "le" in "l'honneur." Additionally, determining when multiword expressions (MWEs) like "New York" or "rock 'n' roll" should be treated as single words presents further difficulty. These issues highlight the complexities inherent in accurately segmenting text into meaningful units for NLP tasks.

Slide 33: Word tokenzation and normalization and sentence segmentation

Slide 33 demonstrates word tokenization in NLTK using a custom regular expression pattern from Bird and Loper (2004) to handle complex text structures. The provided regex specifically preserves abbreviations like "U.S.A.", hyphenated words such as "poster-print", currency values like "$12.40", and ellipses as single tokens, while treating punctuation marks as separate entities. This approach ensures accurate segmentation of the input string into meaningful linguistic units for further processing.

Slide 34: Word tokenzation and normalization and sentence segmentation

Word tokenization involves segmenting text into meaningful units, which varies significantly across languages. The Chinese phrase "姚明进入总决赛" demonstrates this ambiguity by yielding three distinct interpretations: three words (Yao Ming, reaches, finals), five words (splitting names and compounds), or seven individual characters. This example highlights the critical challenge of determining the correct granularity when processing text without explicit word boundaries.

Slide 35: Word tokenzation and normalization and sentence segmentation

Word tokenization strategies vary significantly for writing systems that do not separate words with spaces. In Chinese, segmentation is simplified by commonly treating individual characters (hanzi) as tokens, despite linguistic debates on optimal splitting. Conversely, languages like Thai and Japanese require more complex word segmentation tools, typically implemented using supervised neural sequence models.

Slide 36: Word tokenzation and normalization and sentence segmentation

This slide outlines the core components of text preprocessing in Natural Language Processing, specifically word tokenization, word normalization, and sentence segmentation. It further introduces subword tokenization techniques, including Byte Pair Encoding (BPE), while noting their associated limitations. These topics form a structured progression within the broader NLP tokenization module, following foundational concepts like regular expressions.

Slide 37: Word tokenzation and normalization and sentence segmentation

Word normalization addresses the need to avoid creating separate word vectors for semantically similar variations found in tokenized text, such as "That" versus "that," "U.S.A." versus "USA," or "costs" versus "cost." This process ensures that minor differences in capitalization, punctuation, hyphenation, or numerical precision do not unnecessarily fragment the vocabulary representation.

Slide 38: Word tokenzation and normalization and sentence segmentation

Normalization involves deciding whether to standardize word forms, such as through case folding (lowercasing), based on the specific application. While lowercasing aids information retrieval by matching user queries, it risks losing critical distinctions between proper nouns and common words like "Apple" versus "apple." Conversely, applications like machine translation and information extraction often require preserving casing to differentiate concepts, such as "US" versus "us," making normalization decisions strictly application-dependent.

Slide 39: Word tokenzation and normalization and sentence segmentation

Word normalization via lemmatization reduces inflected words to their shared root lemma, such as converting "am," "are," and "is" to "be," or Spanish variants like "quiero" to "querer." While the slide lists a transformation of "He is reading detective stories" to "He be read detective story," the professor clarifies that the lemmatized result should be "he reads detective stories." Additionally, sentence segmentation identifies sentences within running text as essential NLP units, often utilizing punctuation and regular expressions for this purpose.

Slide 40: Word tokenzation and normalization and sentence segmentation

Slide 40 outlines the upcoming topics on word tokenization, normalization, and sentence segmentation within the broader context of NLP tokenization. The agenda further includes subword tokenization, Byte Pair Encoding (BPE), and the limitations of subword approaches before concluding with a summary. No specific definitions or detailed explanations for these concepts are provided in the source material for this slide.

Slide 41: Word tokenzation and normalization and sentence segmentation

Sentence segmentation splits text by punctuation; while exclamation and question marks are mostly unambiguous sentence boundaries, periods are highly ambiguous due to abbreviations (e.g., "Inc.") and numbers (e.g., "4.3"). Resolving this requires a two-step procedure: first, classify each period as either part of a word or a sentence boundary using rules, regex, abbreviation dictionaries, or machine learning. Second, segment the sentences based on these classifications.

Slide 42: Subword tokenization

Slide 42 presents the outline for subword tokenization, listing it as a key topic following word tokenization, normalization, and sentence segmentation. The slide indicates that the section will cover Byte Pair Encoding (BPE) and address the limitations of subword tokenization before concluding with a summary. No specific definitions or detailed explanations are provided in the source material for this outline slide.

Slide 43: Subword tokenization

Slide 43 introduces subword tokenization as a specific level within the broader context of NLP tokenization. The slide content identifies this concept alongside the general topic of tokenization levels but provides no further definitions, mechanisms, or examples in the available source material.

Slide 44: Subword tokenization

Subword tokenization determines subwords in a data-driven way using compression algorithms like Byte-Pair Encoding (BPE), Unigram language models, and WordPiece. This approach balances vocabulary size by including characters to avoid out-of-vocabulary issues while representing frequent sequences as partial words, such as splitting "tokenizing" into "token" and "izing." The process involves two components: a token learner that induces a vocabulary from a training corpus and a segmenter that tokenizes raw test sentences based on that vocabulary.

Slide 45: Subword tokenization

Slide 45 presents the "Subword tokenization" section within a broader outline covering NLP tokenization topics, including word tokenization, normalization, sentence segmentation, and Byte Pair Encoding (BPE). The slide also lists subsequent points on subword tokenization limitations and a summary. No specific definitions, mechanisms, or examples for subword tokenization are provided in the available source material for this section.

Slide 46: Subword tokenization

Subword tokenization using the Byte Pair Encoding (BPE) algorithm begins with a vocabulary of individual characters. The process iteratively identifies the two most frequently adjacent symbols in the training corpus, merges them into a new symbol added to the vocabulary, and replaces all occurrences of that pair. This repetition continues until a predefined number of merges, k, is reached, which directly determines the final vocabulary size.

Slide 47: Subword tokenization

Most subword algorithms operate within space-separated tokens to ensure efficiency. Before training, a special end-of-word symbol (e.g., '_') is appended to mark word boundaries, and words are separated into individual letters. Crucially, bigram counting is restricted to adjacent characters inside these word boundaries; excluding cross-boundary pairs prevents the vocabulary from exploding in size.

Slide 48: Subword tokenization

Slide 48 illustrates subword tokenization using a Byte Pair Encoding (BPE) learner on a corpus containing words like "low," "lowest," and "newer." The process begins with an initial vocabulary of individual characters, including letters and the underscore symbol. This example demonstrates how BPE operates within NLP tokenization by starting from character-level units before merging them into subword tokens based on frequency.

Slide 49: Subword tokenization

Subword tokenization utilizes the Byte Pair Encoding (BPE) algorithm to learn a vocabulary from a corpus by iteratively merging frequent character pairs. The process begins with individual characters and special tokens, then performs merges such as combining "e" and "r" into "er," followed by merging "er" with "_" to form "er_". This approach constructs subword units that balance the coverage of rare words with the efficiency of common patterns.

Slide 50: Subword tokenization

Slide 50 illustrates Byte Pair Encoding (BPE) subword tokenization using a corpus containing words like "lowest" and "newer." The process begins with a vocabulary of individual characters and specific subunits such as "er," "er_," and "ne." It highlights the third merge step, where the characters 'n' and 'e' are combined to form the new token "ne."

Slide 51: Subword tokenization

Slide 51 illustrates the Byte Pair Encoding (BPE) token learner process through a sequence of merges expanding the vocabulary. Starting with individual characters and basic pairs like "ne" and "er", the algorithm iteratively combines frequent adjacent tokens, such as merging "lo" and "w" to form "low". This progression demonstrates how BPE constructs subword units by repeatedly adding new merged pairs to the vocabulary list.

Slide 52: Subword tokenization

To segment test data using Byte Pair Encoding (BPE), apply the learned merges greedily and strictly in the order they were discovered during training. This sequential process combines characters into subwords, such as merging "e" and "r" to "er," then "er" with a space symbol. Consequently, "newer_" tokenizes as a single word, while "lower_" splits into two tokens ("low" and "er_") because the specific merge for "lo" was not learned or applied in that sequence.

Slide 53: Subword tokenization

Slide 53 presents an outline for the topic of subword tokenization within the broader context of NLP tokenization. The agenda covers foundational concepts, word-level processing including normalization and sentence segmentation, and specific subword methods such as Byte Pair Encoding (BPE). Additionally, the slide indicates that the limitations of subword tokenization will be addressed before concluding with a summary.

Slide 54: Subword tokenization

Subword tokenization determines tokens based on frequency rather than linguistic meaning, which is a key limitation of the approach. As illustrated by an example result from GPT-4o, the resulting subwords are not necessarily linguistically meaningful units. This method prioritizes statistical occurrence over grammatical or semantic structure when breaking text into tokens.

Slide 55: Subword tokenization

Subword tokenization determines units based on frequency rather than linguistic meaning, making it suboptimal for morphologically complex languages. Additionally, multilingual LLM vocabularies are dominated by English, causing non-English languages to generate longer sequences that are more costly and less accurate.

Slide 56: Subword tokenization

Subword tokenization addresses limitations of character and word-level approaches by balancing vocabulary size against sequence length, avoiding the out-of-vocabulary problem inherent in word-level methods. While subwords are primarily determined by frequency rather than linguistic meaning, they remain suboptimal for morphologically complex languages. Additionally, current multilingual models often exhibit English-dominated vocabularies, resulting in longer, less accurate sequences for non-English languages.

Slide 57: Subword tokenization

Subword tokenization addresses limitations found in alternative approaches such as character-based, byte-based, and pixel-based models. While character and byte methods result in excessively long sequences, pixel-based representations currently lag behind subwords in accuracy. Researchers are actively working to bridge this performance gap between pixel-based methods and subword tokenization.

Slide 58: Subword tokenization

Subword tokenization and its alternatives represent a research area that was long ignored but is now increasingly active. This field recently established its own workshop, TokShop, to address these topics. For further exploration, invited talks by Yuval Pinter on the evolution of tokenization and Desmond Elliott on pixel-based models are available online as video resources.

Slide 59: Summary

This slide summarizes the lecture on NLP tokenization by outlining key topics including regular expressions, word tokenization, normalization, and sentence segmentation. It further covers subword tokenization methods like Byte Pair Encoding (BPE) and discusses their limitations. The content serves as a final recap of the module's core concepts before concluding the session.

Slide 60: Summary

Regular expressions remain a vital tool for language processing, specifically for pre-processing, normalization, and sentence segmentation before subword tokenization. While previously the primary method for tokenization, they are now often used in large language models like GPT-2 and GPT-4 despite being language-specific. Subwords, learned via data-driven compression algorithms, offer an alternative but still face limitations that represent an ongoing open challenge in the field.

Slide 61: Summary

Slide 61 serves as a reference list summarizing key literature on NLP tokenization. It cites foundational works such as Jurafsky and Martin's textbook, the NLTK toolkit, and seminal papers on subword units, byte-pair encoding, and character-level translation. The references also include recent studies addressing cross-lingual fairness, domain adaptation, and the cost implications of tokenization in commercial language models.

Key points to remember

The lecture further transitions to word normalization and subword tokenization, distinguishing lemmatization (reducing inflected words to a root lemma) from the data-driven approach of Byte Pair Encoding (BPE). BPE is described as an iterative algorithm that starts with individual characters and repeatedly merges the most frequent adjacent symbol pairs until a target vocabulary size is reached, creating tokens based on frequency rather than linguistic meaning. This method balances vocabulary size by avoiding out-of-vocabulary issues while representing frequent sequences efficiently, though it faces challenges regarding morphological complexity and multilingual bias, where non-English languages often result in longer token sequences. The session concludes by positioning Regex as essential for pre-processing before subword tokenization and noting that tokenization remains an active research field with ongoing developments in cross-lingual fairness and alternative methods like pixel-based models.